



Le monde du développement selon Microsoft

Si vous voulez construire un système d'information, il faut choisir une technologie et naturellement vous allez vous tourner vers Windows, Linux, NET, Java, PHP ou Node.JS et JavaScript : c'est la tendance du moment. Si vous avez de l'expérience dans le monde Windows, vous serez tenté de partir sur la technologie NET. C'est un bon choix qui a fait ses preuves. Pour rappel, NET existe depuis 2001.

Le développement selon Microsoft

Microsoft envoie des messages parfois complexes sur le développement. En voici les fondamentaux :

- La technologie est le Microsoft .NET Framework
- L'environnement de développement est Visual Studio
- Le langage phare est C#
- Le développement mobile se fait avec Xamarin
- Les API pour le développement Desktop sont WinForms et WPF
- Les API pour l'accès aux données sont ADO.NET et Entity Framework
- La technologie web est ASP.NET MVC et ASP.NET web API
- Microsoft décline .NET en .NET Core et ASP.NET Core pour Windows, Linux et macOS
- Le futur du développement est le Cloud Azure et l'intégration des services managés dans les applications

L'exemple officiel, eShopOnContainers

eShopOnContainers est l'exemple officiel du monde du développement Microsoft. Il met en œuvre une application Mobile, une application web, une app PWA, une application ASP.NET MVC, des web API et des micro-services et une API Gateway. La communication entre les micro-services est assurée par Event Bus ou RabbitMQ, au choix. ¹

L'envers du décor

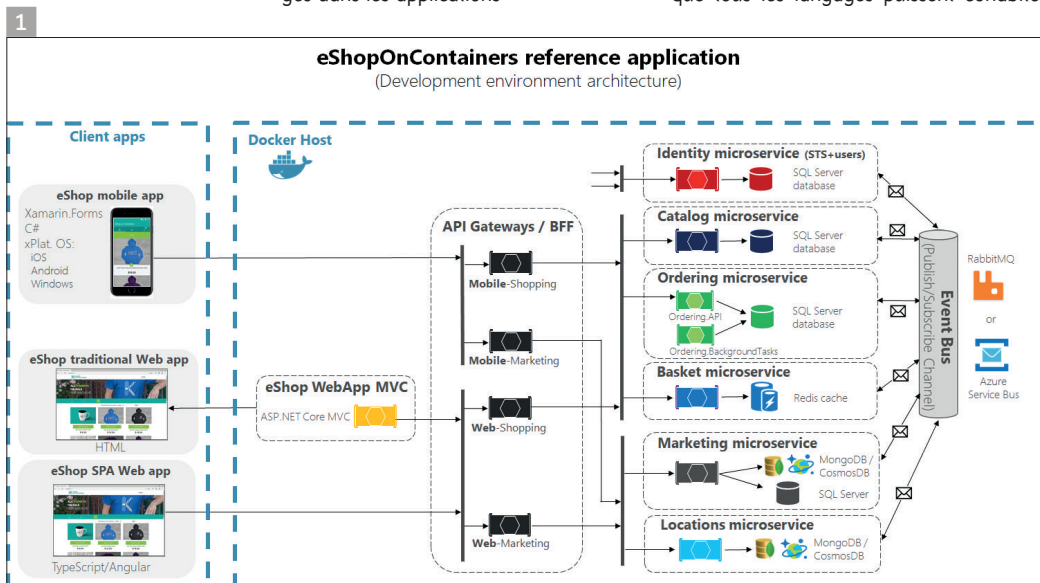
Le développement *from scratch* est plutôt évident à comprendre, mais que faire des dizaines d'applications existantes ? Doit-on les réécrire (scénario impossible) ou les adapter.

Que devons-nous faire de nos vieilles applications WinForms, SilverLight (RIA ou Rich Internet Applications) ? Dans certains scénarios, on va fabriquer des API au-dessus des systèmes existants. Des API REST pour que tous les langages puissent cohabiter.

Le JSON est moins verbeux que le XML ; c'est du pareil au même dans la philosophie. L'ouverture aux multiples langages est la tendance actuelle. La révolution JavaScript est passée par là, à la fois côté client et côté serveur avec NodeJS.

Application web ou application Desktop ?

Il est de mode de faire des applications web et même full JS/TS web avec Angular, React ou Vue.JS. Ce n'est pas la panacée, car ces applications sont longues et coûteuses à développer pour une complexité exponentielle. Le langage TS utilisé par Angular est OOP, mais les développeurs font beaucoup de copier/coller et rendent ce code spaghetti. C'est le principal problème des technologies JavaScript. À titre personnel, j'émet des réserves sur les applications full JavaScript, car elles sont difficilement maintenables et truffées de copier/coller. De plus, dès que le back-end web API change, c'est souvent la galère pour répercuter les divers changements. Pour finir, Angular est d'une complexité délirante et le coût en formation est non négligeable pour une technologie qui « bouge » tous les ans. Je préfère de loin les applications web Server de type ASP.NET MVC qui permettent de séparer le code et les pages dans des modules distincts et qui ne sont pas des amas de code statique. ASP.NET MVC est beaucoup plus puissant que les applications de type Angular/React et n'adresse pas les mêmes problèmes. Une autre solution est de réaliser des applications Desktop WinForms ou WPF. La technologie WinForms n'est pas dépréciée, n'en déplaie à certains, car elle est rapide. WPF est toujours présent. UWP existe toujours malgré le faible taux d'adoption. Il y a moins de 2% des applications



faites en UWP (source Télémétrie Visual Studio de Microsoft). Un détail qui a son importance. Pour tout vous dire, il n'y a que Microsoft qui fait des applications UWP. Et encore, c'est C++, comme le Windows Terminal ou les accessoires (Calc, Paint3D, Courrier, etc.).

La révolution XAML avec WinUI

Malgré le fait que Windows n'utilise aucune de ses deux technologies WinForms et WPF, il existe un nouveau venu dans le domaine qui se nomme WinUI qui représente les contrôles XAML natifs de Windows. Il est fortement conseillé par Microsoft d'utiliser ces contrôles natifs dans les applications WinForms ou WPF ou même C++. Le style minimaliste est de mise. L'ancien nom Métro vous rappelle quelque chose ? Microsoft abandonne progressivement WPF, car trop lent, trop lourd et le modèle WPF est complexe pour un ratio élégance/productivité très limité. Si vous le connaissez déjà, vous ne perdez pas vos connaissances XAML, mais vous ferez du WinUI. XAML n'est plus associé à Silverlight, RIA ou WPF, mais à Windows.

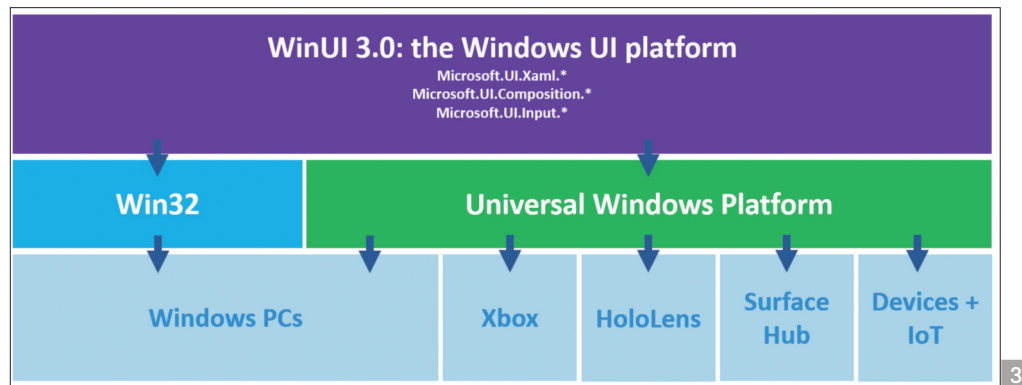
2

La revanche de Windows sur NET/WPF

C'est encore la guerre Windows Division / Developer Division qui se traduit avec WinUI. Le traumatisme de Longhorn est toujours dans les têtes chez Microsoft. Cette initiative qui consistait à vouloir introduire du C#/NET dans Windows avec WinFS, un système de fichier orienté objet, Avalon (WPF) et WCF (Windows Communication Foundation) et WF (Windows Workflow). Microsoft a décidé de terminer cette expérience et n'a jamais utilisé de C#/NET dans Windows. Le seul module dans Windows qui utilise du NET est une partie du serveur web IIS pour la gestion ASP.NET. Sinon rien du tout ! Sur 4000 dlls dans System32, tout est natif. Mieux, le code a évolué du C++03 au C++ Moderne et les modules Shell standard des accessoires et du control panel sont faits en XAML avec les contrôles natifs, mais toujours en C++.

WinUI 3.0

Revenons à WinUI, voici la roadmap disponible sur GitHub : <https://github.com/microsoft/>



3

microsoft-ui-xaml/blob/master/docs/roadmap.md

WinUI 3 étendra considérablement la portée de WinUI pour inclure la plateforme d'interface utilisateur native Windows 10 complète, qui sera désormais entièrement découplée du SDK UWP.

Nous nous concentrons sur l'activation de trois cas d'utilisation principaux :

- Modernisation des applications Win32 existantes
 - Vous permettant d'étendre les applications Win32 (WPF, WinForms, MFC ..) existantes avec une interface utilisateur Windows 10 moderne à votre rythme en utilisant la dernière version à venir de Xaml Islands
 - Création de nouvelles applications Windows
 - Vous permettant de créer facilement de nouvelles applications Windows modernes "à la carte" avec votre choix de modèle d'application (Win32 ou UWP) et de langue (.NET Core ou C++)
 - Activation d'autres Frameworks
 - Fournir l'implémentation native pour d'autres frameworks comme React Native lors de l'exécution sur Windows
- WinUI 3.0 est en release Alpha (novembre 2019)

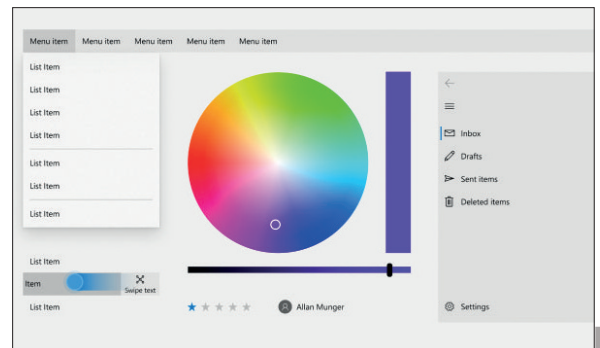
Présentation conceptuelle de WinUI 3

3

Plateforme WinUI 3

Vous pouvez regarder la session de conférence Build 2019 State of the Union : The Windows Presentation Platform. Pour plus de détails : <https://mybuild.techcommunity.microsoft.com/sessions/77008>

Les API UWP Xaml existantes livrées avec le système d'exploitation ne recevront plus de nouvelles mises à jour de fonctionnalités. Elles recevront toujours des mises à jour de sécurité et des correctifs critiques en



2

fonction du cycle de vie de la prise en charge de Windows 10. La plateforme Windows universelle contient plus que le Framework Xaml (par exemple, le modèle d'application et de sécurité, le pipeline de médias, les intégrations de shell Xbox et Windows 10, la prise en charge étendue des appareils) et continuera d'évoluer. Toutes les nouvelles fonctionnalités Xaml seront développées et livrées à la place dans WinUI.

Avantages de WinUI 3

WinUI 3 offrira un certain nombre d'avantages par rapport au Framework UWP Xaml actuel, WPF, WinForms et MFC, ce qui fera de WinUI le meilleur moyen de créer l'interface utilisateur de l'application Windows :

1° La plateforme d'interface utilisateur native de Windows

WinUI est la plateforme d'interface utilisateur native hautement optimisée utilisée pour créer Windows lui-même, désormais plus largement disponible pour tous les développeurs à utiliser pour atteindre Windows. Il s'agit d'une plateforme d'interface utilisateur entièrement testée et éprouvée qui alimente l'environnement du système d'exploitation et les expériences essen-

.NET – A unified platform



4

tielles de plus de 800 millions de PC Windows 10, Xbox One, HoloLens, Surface Hub et d'autres appareils.

2° La dernière conception Fluent

WinUI est l'objectif principal de Microsoft pour l'interface utilisateur et les contrôles Windows natifs et accessibles et est la source définitive pour le Fluent Design System sur Windows. Il prendra également en charge les dernières innovations de composition et de rendu de niveau inférieur telles que les animations vectorielles, les effets, les ombres et l'éclairage.

3° Développement de bureau "à la carte" plus facile

WinUI 3 vous permettra de mélanger plus facilement et de faire correspondre la bonne combinaison de :

- Langage : .NET (C #, Visual Basic), C ++
- Modèle d'application: UWP, Win32
- Emballage : MSIX, AppX pour le Microsoft Store, non emballé
- Interop : utilisez WinUI 3 pour étendre les applications WPF, WinForms et MFC existantes avec une interface utilisateur Fluent moderne

4° Compatibilité descendante pour les nouvelles fonctionnalités

Les nouvelles fonctionnalités WinUI continueront d'être rétro-compatibles avec un large éventail de versions de Windows. Vous pouvez commencer à créer et à envoyer des applications avec de nouvelles fonctionnalités dès qu'elles sont publiées, sans avoir à

attendre que vos utilisateurs mettent à jour Windows.

5° Prise en charge du développement natif

WinUI peut être utilisé avec .NET, mais ne dépend pas de .NET : WinUI est 100% C ++ et peut être utilisé dans des applications Windows non gérées, par exemple en utilisant le standard C ++ 17 via C ++ / WinRT.

6° Mises à jour plus fréquentes

WinUI continuera à livrer de nouvelles versions stables 3 fois par an, avec des versions préliminaires mensuelles.

7° Développement open source et engagement communautaire

WinUI continuera d'être développé en tant que projet open source sur GitHub. WinUI 2 est déjà open source dans le dépôt, et il est prévu d'ajouter le framework WinUI 3 Xaml complet.

Vous pouvez vous engager directement avec l'équipe d'ingénierie principale de Microsoft et contribuer aux rapports de bogues, aux idées de fonctionnalités et même au code : consultez le Guide de contribution pour plus d'informations. Vous pouvez également essayer les versions préliminaires mensuelles pour voir les nouvelles fonctionnalités en cours de développement et aider à façonner leur forme finale.

8° Une cible Windows native pour les frameworks web et multiplateforme

WinUI 3 est mieux optimisé pour les biblio-

thèques et les frameworks sur lesquels s'appuyer.

Par exemple, il est prévu de baser la nouvelle implémentation C ++ React Native Windows hautes performances sur WinUI 3.

La roadmap NET 5

On nous annonce NET 5 pour fin 2020. C'est la fusion entre .NET Framework 4.8 et .NET Core 3.x. Pourquoi pas. Avec en plus, le multiplateforme sous Linux, Mac et Windows et pour le mobile sous Android et iOS. Mono permettant toujours de faire du GTK sous Gnome. NET 5 est avant tout une plateforme unifiée. On y trouve WinRT qui est disponible depuis Windows 8, les compilateurs Roslyn C# et VB.NET, les langages C#, VB.NET et F# et C ++/CLI et on y trouve plusieurs domaines :

- Le desktop avec WPF, WinForms et UWP
- Le web avec ASP.NET
- Le cloud avec Azure
- Le mobile avec Xamarin
- Le gaming avec Unity
- L'IoT
- L'AI avec ML.NET

On dispose des outils Visual Studio (Windows et Mac) et Visual Studio Code. Visual Studio est l'outil dans lequel on fait tout. Si vous avez la possibilité, passez à la version Pro (ou Enterprise).

4

L'accès aux données

L'accès aux données selon Microsoft a été pendant 20 ans MDAC (Microsoft Data Access Components) composé de ADO, RDO, OLEDB et ODBC. Avec .NET, on a vu apparaître ADO.NET puis Entity Framework (EF). EF est un ORM : attention. Cela peut être une catastrophe. Les requêtes générées sont complexes et parfois horribles. C'est un ORM et donc comme tout ORM, c'est une technologie client qui essaie de s'affranchir du SQL, ce qui est une hérésie. Les jeunes développeurs pensent que c'est cool, car ils ne font que du C# et pas de SQL. Le résultat est souvent décevant et catastrophique. Anecdote de terrain : une entreprise a failli mettre la clé sous la porte après avoir fait son SI en ASP.NET/EF, car les pages mettaient 10 secondes à s'afficher. Un audit a été réalisé et les requêtes EF étaient fautes. Le DSI a été viré et les développeurs licenciés. De nombreux architectes bannissent l'emploi de EF sur les pro-

jets et proposent plutôt de l'ADO.NET avec Dapper, une librairie open source fortement typée. On conseillera l'utilisation traditionnelle des projets SQL Server ou Oracle avec des traitements serveur... SQL ou procédures stockées. Amis développeurs, apprenez le SQL, c'est important. Linq to SQL c'est bien quand il y a dix enregistrements qui se battent en duel dans une table sinon ça ramène toute la base en local et les ingénieurs systèmes vont vous détester.

Le monde web ASP.NET

ASP.NET Core introduit de nouveaux paradigmes. En effet, la plateforme NET Core tourne sous Windows, Linux et macOS. Quand on dit Linux, cela ouvre le champ des serveurs Nginx Alpine, CentOS, RHEL, Debian, etc. Bref, le web version « Microsoft seulement sur IIS » est révolu.

C'est un sacré pavé dans la marre pour les développeurs Windows. Il va falloir installer Windows Services for Linux v2 (WSL2) sous Windows 10 au mieux, avoir une seconde machine sous Linux ou bien jouer avec des containers Docker. Un développeur a besoin de confort donc je serais vous, je préparerais moralement votre patron à vous payer un second laptop... C'est indispensable.

Cela implique que les développeurs Windows doivent maintenant apprendre Linux (le bash, TCP/IP, la sécurité, etc.). Celui qui reste que sous Windows sera dans une voie de garage à terme. Le Back-end sera de plus en plus sous Linux, car plus léger que Windows Server. SQL Server existe sous Linux donc vous ne serez pas perdu. Amis développeurs, faites votre plan de formations et avancez vos pions. Ne passez pas à côté de ce virage.

Côté nouveautés, ASP.NET Core introduit web API Core, ASP.NET MVC Core et gRPC ainsi que les briques pour l'authentification. ASP.NET Core & MVC Core sont des technologies d'avenir pour le développeur Microsoft. Vous pouvez miser dessus et vous certifier.

L'aspect économique

Ne vous y trompez pas, les bijoux de Microsoft ce sont Windows, Office 365 et Azure. Tout est bon pour vous faire venir sur Azure. On vous parle de VM Linux, de containers, de Docker, de Kubernetes et du support de beaucoup de technologies



Timothé LARIVIERE | Tech Lead chez Infeeny
timothe.lariviere@infeeny.com | timothelariviere.com

LE DÉVELOPPEMENT MOBILE AVEC XAMARIN

La promesse de Xamarin est alléchante : réutiliser vos compétences .NET pour réaliser des applications mobiles en C# (ou F#) et en partageant une majeure partie du code entre les plateformes, possible grâce à Mono qui est disponible sur les OS d'Apple et Linux ! Cela donne un avantage certain face au développement natif. Vous n'êtes pas contraint de redévelopper vos applications plusieurs fois pour les différentes plateformes (Android + Java, iOS + Swift) avec tous les risques que cela implique (bugs, pas équivalent niveau fonctionnalités, etc.). Vous pouvez partager votre code dans des DLLs (.NET Standard 2.0 par ex.) et bénéficier de tout l'existant .NET (notamment NuGet) pour vous faciliter la vie, réduire les

temps de développement et le nombre de développeurs nécessaires.

Cependant, cela ne vous dispense pas d'apprendre les spécificités des plateformes que vous ciblez.

Si vous faites du Xamarin dit natif, vous aurez à créer les interfaces graphiques en utilisant la manière de faire de la plateforme (Activity / XML, UIController / Storyboard, etc.). Il faudra alors du temps aux équipes pour se former, car il n'est pas aisé d'apprendre une nouvelle plateforme. Xamarin.Forms est un peu plus flexible sur ce point. Le développement des interfaces graphiques se faisant à travers un XAML très semblable à WPF et UWP. Xamarin.Forms se chargera lui-même de faire la

correspondance entre le XAML et les contrôles natifs de la plateforme au runtime.

Le pourcentage de code partagé variera considérablement selon le type d'application que vous faites. Une application orientée métier pourra maximiser son partage en centralisant les règles métiers. Une application B2C aura des besoins graphiques beaucoup plus importants, pas forcément partageables. Dans la pratique, cela sera plus de l'ordre de 60-40, assez loin des 80-20 promis par le marketing. Pour conclure, le développement mobile avec Xamarin est très intéressant, mais il faut bien étudier vos besoins et vos équipes avant de décider de l'utiliser au lieu du développement natif.

Open-Source. La réalité c'est que vous venez comme vous êtes (comme au Mc Do) et que vous allez un ou l'autre utiliser et consommer de l'Azure ! C'est une question de temps. Le virage technologique est enclenché. Le Cloud s'impose.

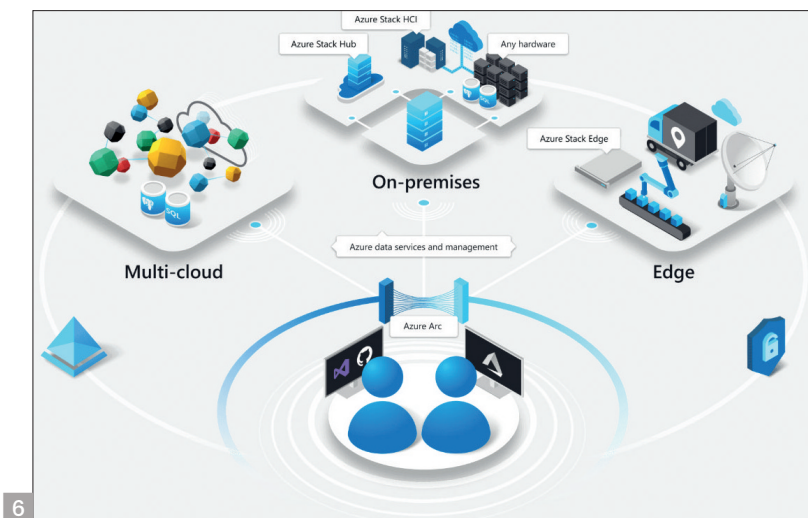
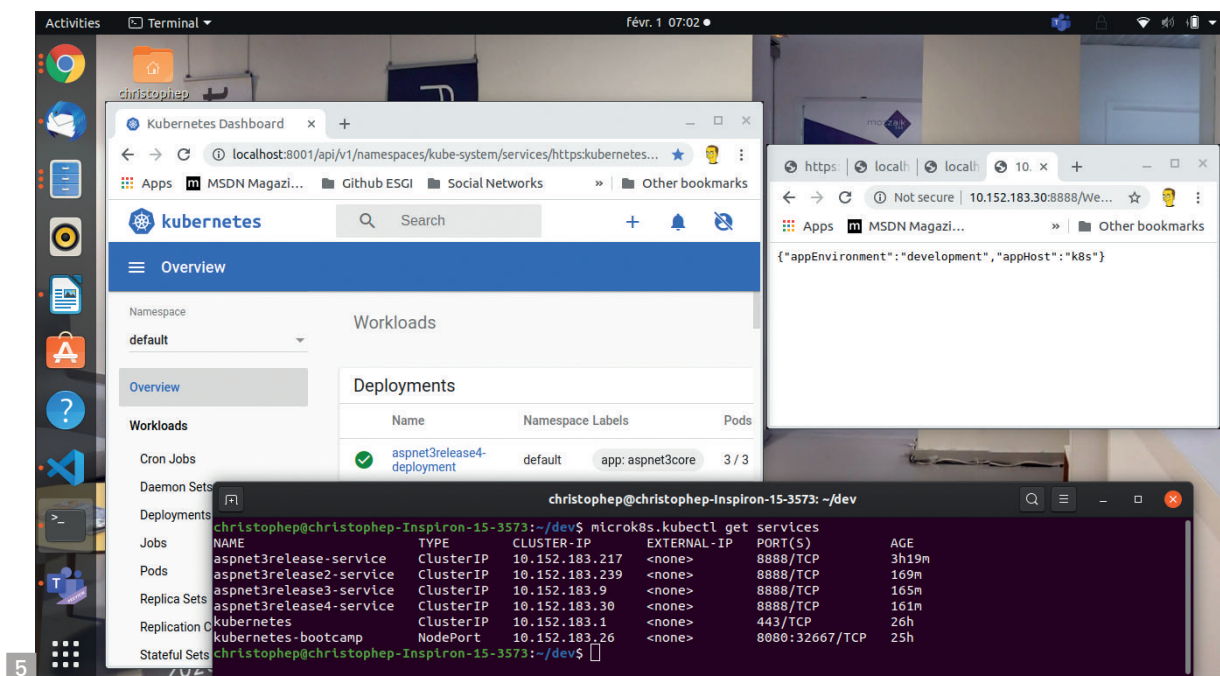
La modernisation des applications

Il est un domaine dans lequel les entreprises de services se sont engouffrées : la modernisation des applications. Sous ce nom exotique, on trouve la mise en œuvre de technologies Cloud comme la télémétrie, le stockage et l'appel à des services managés de type web API en REST. Une application Desktop devient connectée au service de l'entreprise, mais aussi à Azure pour son monitoring, son logging et utiliser de plus en plus de services.

Cela passera par du middleware comme Service Bus ou RabbitMQ, SQL Azure ou peut-être Service Fabric ou même de la technologie Serverless comme Azure Functions. Il paraît que 2020 est l'année du Serverless... À suivre.

Microservices, Docker et Kubernetes

Les architectures microservices sont l'avenir du développement en entreprises. Les microservices sont indépendants, autosuffisants et possèdent leur base SQL ou NoSQL et communiquent via un Bus de Message comme Azure Service Bus ou RabbitMQ. Les microservices ouvrent la voie à l'utilisation des plateformes d'Api Managements ou Gateways d'API et de cluster de pods Docker avec un orchestrateur comme Kubernetes. L'environnement



Azure propose AKS et Microsoft fait l'éloge de AKS et Kubernetes matin, midi et soir sauf que k8s (son petit nom) ne tourne pas sous Windows, mais sous Linux... Si vous voulez apprendre k8s, prenez un PC sous Linux Ubuntu 19.10 et installez Docker et MicroK8s pour maîtriser la bête. C'est gratuit et ça fait 2x30 MB et c'est gratuit.

Je vous donne rendez-vous le mois prochain pour un article spécial k8s sous Linux.

5

Azure Hybrid

Avec les scénarios d'entreprise hybrides, Microsoft nous prépare Azure Arc qui

consiste à mettre le monde du développement et de l'infrastructure dans un rôle hybride. Les connexions se font *on-premises* et sur le Cloud, voire chez différents vendeurs de Cloud. Les outils de développement évolueront, c'est certain et Visual Studio aura de nouvelles fonctionnalités à n'en pas douter. 6

Conclusion

Le développement selon Microsoft c'est la plateforme .NET, Azure et Windows. Ce qui est troublant c'est que Microsoft fait 95% de ses produits en C++ et rien en .NET à part Visual Studio et ses composants NE... Donc là où Microsoft il y a 20 ans faisait du

Dog-fooding en construisant des SDK et en les utilisant dans ses produits, on est passé à l'ère du Marketing où l'on pousse des technologies que l'on n'utilise pas en interne ou si peu.

Est-ce bien raisonnable ? Oui la technologie est fiable, mais Windows ne contient aucun composant NET à part la redistribution du Framework dans le répertoire Windows. À quand le grand saut ? Pas pour si tôt. En tant que fervent adepte du natif, je souhaite que Windows continue de s'ouvrir avec les API WinRT pour laisser le choix aux développeurs d'utiliser le langage de leur choix.

Si NET pouvait être modifié pour que les modules soient en mode back-end natif, cela permettrait de clore de nombreux débats.

C'est le modèle de Go, Apple avec Objective-C, Rust, etc. Ils ont des back-end LLVM et c'est du natif. Si le C#/NET pouvait faire du natif avec autre chose qu'une gestion de la mémoire via un Garbage Collector, je suis certain que Microsoft Windows Core s'ouvrirait à C#/NET. C'est un souhait personnel, mais qui a peu de chance d'aboutir, car cela casse l'existant. Pourtant cela irait dans le sens des autres acteurs du marché. Microsoft n'invente rien : il suit et s'adapte. Le message est passé.